

PySpark for Data Engineering — Zero to Advanced

Complete Study Guide: Notes + Practice + Interview Q&A

Background assumed: You know Python and SQL, and you're new to Spark. **How to use this guide:** Read the notes for a topic, try the practice exercise yourself (don't just read the code — type it and run it), then check your understanding against the interview Q&A. Go module by module; don't skip Module 1 even if it feels "too basic" — the mental model it builds is used everywhere else.

Module 1 — Spark Fundamentals

1.1 What Spark is, and why it exists (vs pandas, vs Hadoop MapReduce)

The problem Spark solves: pandas is fast and easy, but it runs on **one machine** and loads all data **into memory**. The moment your data is bigger than your RAM (say, 500 GB of logs), pandas simply cannot do the job. You need something that can split the data and the work across many machines.

Hadoop MapReduce was the first popular answer to this. It processed huge datasets across clusters, but every step wrote intermediate results to disk. Disk is slow. Multi-step pipelines (very common in real ETL) became painfully slow because of constant disk I/O between steps.

Spark's big idea: keep data **in memory** across the cluster as much as possible, and build a chain of operations that Spark only executes when needed (this is "lazy evaluation" — more below). This makes iterative and multi-step jobs dramatically faster than MapReduce, while still scaling to huge datasets like Hadoop does.

	pandas	Hadoop MapReduce	Spark
Runs on	1 machine	Cluster	Cluster
Data location	RAM only	Disk between steps	RAM (preferred), spills to disk
Ease of use	Very easy	Verbose, low-level Java	Easy, Python/SQL/Scala APIs
Speed on multi-step jobs	N/A (small data)	Slow	Fast

Mental model: Spark = "pandas-like operations, but the engine automatically splits your data and your computation across many machines, and is smart about avoiding unnecessary disk writes."

1.2 Cluster architecture: Driver, Executors, Cluster Manager

When you run a Spark job, three kinds of processes are involved:

- **Driver** — the "brain." This is where your code (the `main()` / your script) actually runs. It builds the execution plan, and it hands out tasks. It also collects final results when you call something like `.collect()`.
- **Executors** — the "workers." These run on the cluster's machines. Each executor runs the actual tasks (reading data, filtering, aggregating) and holds cached data in memory. A cluster typically has many executors running in parallel.
- **Cluster Manager** — the "resource allocator." It decides which physical machines the driver and executors get to run on. Common cluster managers: **YARN**, **Kubernetes**, **Mesos**, or Spark's own **Standalone** manager. In Databricks, this is managed for you.

Flow: You submit a job → the Cluster Manager gives your Driver some machines → the Driver asks the Cluster Manager for Executors → the Driver splits work into **tasks** and sends them to Executors → Executors do the work and report back.

1.3 SparkSession, SparkContext

- **SparkContext** — the original, low-level entry point to Spark (mainly used for RDDs). You rarely create this directly anymore.
- **SparkSession** — introduced in Spark 2.0, this is the **single unified entry point** for everything: DataFrames, SQL, streaming, and configuration. In modern PySpark, you almost always start your code with:

```
python

from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("MyFirstApp")
    .master("local[*]") # run locally using all CPU cores; omit this on a real
    .getOrCreate()
)
```

`SparkSession` internally creates a `SparkContext` for you (accessible via `spark.sparkContext` if you ever need it).

1.4 Lazy evaluation, transformations vs actions

This is one of the **most important concepts** in Spark — interviewers ask about it constantly.

- **Transformations** — operations that describe *what* you want done, but don't execute immediately. Examples: `.select()`, `.filter()`, `.withColumn()`, `.groupBy()`. They return a **new DataFrame** and just add a step to Spark's execution plan.
- **Actions** — operations that actually **trigger execution** of the whole chain of transformations. Examples: `.show()`, `.collect()`, `.count()`, `.write()`.

python

```
df2 = df.filter(df.age > 18)           # transformation - nothing runs yet
df3 = df2.select("name", "age")       # transformation - still nothing runs
df3.show()                           # ACTION - now Spark actually reads data and
```

Why laziness matters: Spark sees your *entire* chain before running anything, so it can optimize the whole plan (e.g., push filters down close to the data source, skip columns you never selected) instead of executing each line naively, one at a time.

1.5 DataFrames vs RDDs (why DataFrames win for data engineering)

- **RDD (Resilient Distributed Dataset)** — the original, lowest-level Spark data structure: a distributed collection of Python/Java/Scala objects. You write operations as plain functions (`.map()`, `.filter()` with lambdas). Spark has **no idea** what's inside these objects, so it can't optimize much.
- **DataFrame** — a distributed collection of data organized into **named columns**, like a SQL table. Because Spark *knows the schema*, the **Catalyst optimizer** (Module 5) can rewrite and optimize your query automatically, and the **Tungsten engine** can use highly efficient memory layouts.

Rule of thumb for data engineering: Always prefer DataFrames (or Spark SQL) over RDDs. Only drop down to RDDs for rare cases needing very custom, low-level control that DataFrame APIs can't express.

Practice Exercises — Module 1

1. Install PySpark locally (`pip install pyspark`) and create a `SparkSession`. Print `spark.version`.
2. Create a DataFrame from a small Python list of dictionaries (e.g., 5 people with `name` and `age`). Chain a `.filter()` and a `.select()`, then call `.show()`. Add a `print("before action")` right after the transformations and a `print("after action")` after `.show()` — notice nothing about your data is processed until `.show()` runs.
3. Explain out loud (or in writing), in your own words, why Spark is faster than Hadoop MapReduce for multi-step jobs.

Interview Q&A — Module 1

Q1: What is lazy evaluation in Spark, and why does it matter? A: Spark doesn't execute transformations (`select`, `filter`, `groupBy`, etc.) immediately — it builds up a logical plan (a DAG) of operations. Execution only happens when an action (`show`, `count`, `collect`, `write`) is called. This matters because it lets Spark's Catalyst optimizer see the *entire* chain of operations and optimize it as a whole — e.g., combining filters, pushing predicates down to the data source, and skipping unused columns — rather than executing each step wastefully in isolation.

Q2: What's the difference between a transformation and an action? Give two examples of each. A: Transformations produce a new DataFrame and are lazy (not executed immediately) — e.g. `.filter()`, `.withColumn()`. Actions trigger actual computation and return a result to the driver or write output — e.g. `.count()`, `.collect()`.

Q3: Explain the role of the Driver vs Executors. A: The Driver runs your main program, builds the execution plan (DAG), and schedules tasks. Executors are worker processes on the cluster that actually run those tasks against the data and hold cached data in memory. The Driver coordinates; the Executors do the heavy lifting.

Q4: Why do DataFrames generally outperform RDDs? A: DataFrames carry schema information, which lets Spark's Catalyst optimizer analyze and rewrite queries (predicate pushdown, column pruning, join reordering) and lets the Tungsten engine use compact, efficient binary memory representations. RDDs are opaque, unstructured collections of objects — Spark can't "see inside" them, so none of these optimizations apply.

Q5: What is a SparkSession, and how does it relate to SparkContext? A: SparkSession (Spark 2.0+) is the single unified entry point for DataFrame, SQL, and streaming APIs. It wraps and manages a SparkContext internally (the older, lower-level entry point mainly used for RDDs), so in modern code you almost never create a SparkContext directly.

Module 2 — DataFrame Core Operations

2.1 Creating DataFrames (from files, from Python data, with schemas)

python

```
# From a Python list of tuples, with explicit column names
df = spark.createDataFrame([("Alice", 30), ("Bob", 25)], ["name", "age"])

# From a file (schema auto-inferred)
df = spark.read.csv("people.csv", header=True, inferSchema=True)

# From a file with an EXPLICIT schema (recommended for production — see Module
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
schema = StructType([
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True),
])
df = spark.read.csv("people.csv", header=True, schema=schema)
```

2.2 select, filter, withColumn, drop, rename

python


```
df.select("name", "age").show()
df.filter(df.age > 18).show()           # or: df.filter("age > 18")
df.withColumn("age_plus_1", df.age + 1).show()  # add/replace a column
df.drop("age").show()
df.withColumnRenamed("name", "full_name").show()
```

Key idea: **every one of these returns a brand-new DataFrame** — Spark DataFrames are immutable. You're never modifying `df` in place; you're building a chain:

```
df.filter(...).select(...).withColumn(...).
```

2.3 Aggregations: groupBy, agg

```
python

from pyspark.sql import functions as F

df.groupBy("department").count().show()

df.groupBy("department").agg(
    F.avg("salary").alias("avg_salary"),
    F.max("salary").alias("max_salary"),
    F.count("*").alias("num_employees"),
).show()
```

`groupBy` alone returns a special `GrouppedData` object — it's not useful until you call an aggregation function (`.count()`, `.agg()`, `.avg()`, etc.) on it.

2.4 Sorting, distinct, limit

```
python

df.orderBy(df.age.desc()).show()      # or df.sort(...)
df.select("department").distinct().show()
df.limit(10).show()
```

2.5 Joins (inner, left, right, outer, semi, anti) and join strategies

```
python

df1.join(df2, on="id", how="inner")    # default
df1.join(df2, on="id", how="left")     # keep all of df1
df1.join(df2, on="id", how="right")    # keep all of df2
df1.join(df2, on="id", how="outer")    # keep everything from both
df1.join(df2, on="id", how="left_semi") # rows in df1 that HAVE a match in df2
df1.join(df2, on="id", how="left_anti") # rows in df1 that have NO match in df
```

- **semi join** — "filter df1 to only rows that exist in df2," without pulling in df2's columns. Useful for existence checks.
- **anti join** — the opposite: "filter df1 to rows that do NOT exist in df2." Very common for finding new/unmatched records in ETL.

Join strategies (how Spark physically executes a join) are covered in depth in Module 5 (broadcast joins vs. shuffle joins) — for now, just know that Spark picks a strategy automatically based on data size, but you can influence it.

Practice Exercises — Module 2

1. Create two small DataFrames: `employees` (id, name, dept_id) and `departments` (dept_id, dept_name). Perform an inner join and show the result.
2. Using the same `employees` DataFrame, find all employees who do **not** have a valid `dept_id` present in `departments` (hint: anti join).
3. Given a DataFrame of sales (`region`, `amount`), compute total and average sales **per region**, sorted by total sales descending.
4. Add a new column `amount_after_tax` that is `amount * 1.18`, using `withColumn`.

Interview Q&A — Module 2

Q1: Why are Spark DataFrames immutable, and what does that mean practically? A:

Every transformation returns a new DataFrame rather than modifying the existing one in place. Practically, this means you build pipelines by chaining calls (`df.filter(...).select(...)`) and must reassign results (`df = df.withColumn(...)`) if you want to "update" a variable — the original DataFrame object is never mutated.

Q2: What's the difference between a left semi join and a left anti join? A: A left semi join returns rows from the left table that **have** a match in the right table, but only includes columns from the left table. A left anti join returns rows from the left table that **do not** have a match — it's effectively a "NOT IN" / "NOT EXISTS" filter. Neither includes columns from the right table.

Q3: What happens if you call `.groupBy("col")` without any aggregation afterward? A: It returns a `GroupedData` object, not a DataFrame — you can't `.show()` it directly. You must chain an aggregation like `.count()`, `.agg()`, `.sum()`, etc., to get back a usable DataFrame.

Q4: You need to remove exact duplicate rows from a DataFrame. What method would you use? A: `.distinct()` for full-row duplicates, or `.dropDuplicates(["col1", "col2"])` to de-duplicate based on a subset of columns (keeping one arbitrary row per group of duplicates).

Q5: What's a subtle danger of using `inferSchema=True` on a large CSV file in production? A: Spark has to make an extra full (or partial) pass over the data just to guess column types, which is slow and can guess wrong (e.g., a column that's mostly integers but has one string value gets typed as string, or dates get inferred incorrectly). In production, you should define an explicit `StructType` schema for reliability and performance (see Module 3).

Module 3 — Reading & Writing Data

3.1 File formats: CSV, JSON, Parquet, ORC, Avro

Format	Type	Notes
CSV	Row-based, text	Human-readable, no schema/types built in, slow for large data, easy to get encoding/delimiter issues
JSON	Row-based, text	Good for nested/semi-structured data, still text (slow, large on disk)
Parquet	Columnar, binary	The default choice for data engineering. Compressed, stores schema, supports predicate/column pushdown (Spark reads only the columns/rows it needs)
ORC	Columnar, binary	Similar to Parquet, common in the Hive/Hadoop ecosystem
Avro	Row-based, binary	Great for streaming and schema evolution (e.g., Kafka messages) since it stores schema with the data

```
python
```

```
df = spark.read.csv("path", header=True, inferSchema=True)
df = spark.read.json("path")
df = spark.read.parquet("path")
df = spark.read.orc("path")
df = spark.read.format("avro").load("path")

df.write.parquet("output_path", mode="overwrite")
```

Why Parquet wins for data engineering: it's columnar (so if you only need 3 of 50 columns, Spark can skip reading the rest), it's compressed (much smaller on disk = faster I/O), and it carries schema metadata (no guessing needed).

3.2 Schema inference vs explicit schemas (StructType)

Inference is convenient for exploration but risky in production (extra read pass, wrong type guesses, silent data quality issues). The professional approach: define the schema explicitly.

```
python
```

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType,

schema = StructType([
    StructField("order_id", IntegerType(), nullable=False),
    StructField("customer", StringType(), nullable=True),
    StructField("amount", DoubleType(), nullable=True),
])

df = spark.read.schema(schema).csv("orders.csv", header=True)
```

With Parquet/Avro, schema is already embedded in the file, so explicit schemas matter less there — mostly a CSV/JSON concern.

3.3 Partitioned writes, write modes

Write modes control what happens if the destination already has data:

- `overwrite` — replace existing data
- `append` — add to existing data
- `ignore` — do nothing if data already exists
- `error` / `errorifexists` — (default) fail if data already exists

python

```
df.write.mode("overwrite").parquet("output_path")
```

Partitioned writes — split output into subfolders based on column values, so future reads can skip irrelevant folders entirely (this is called **partition pruning**):

python

```
df.write.partitionBy("year", "month").mode("overwrite").parquet("output_path")
```

This creates a folder structure like `output_path/year=2024/month=01/...`. A later query filtering `WHERE year = 2024 AND month = 1` will only read that one folder, not the whole dataset — a huge performance win at scale.

Caution: don't over-partition (e.g., partitioning by a high-cardinality column like `user_id`) — it creates thousands/millions of tiny files, which hurts performance ("small file problem").

3.4 Handling nested/semi-structured data (structs, arrays, maps)

python

```
# Access a nested struct field
df.select("address.city", "address.zipcode")

# Explode an array column into multiple rows
from pyspark.sql import functions as F
df.select("name", F.explode("hobbies").alias("hobby"))

# Access a map value
df.select(F.col("attributes")["color"])
```

`explode` is one of the most commonly used functions when flattening JSON-style nested data into a tabular format for analytics.

Practice Exercises — Module 3

1. Write a small DataFrame to disk as both CSV and Parquet, and compare the file sizes.
2. Define an explicit `StructType` schema for a CSV with columns `id (int)`, `name (string)`, `price (double)`, and read the file using it.
3. Write a DataFrame partitioned by a `country` column, then inspect the output folder structure.
4. Create a DataFrame with an array column (e.g., `tags`) and use `explode` to flatten it into one row per tag.

Interview Q&A — Module 3

Q1: Why is Parquet generally preferred over CSV/JSON for large-scale data engineering pipelines? A: Parquet is columnar and binary: it compresses well, stores schema with the data (no inference needed), and supports column pruning and predicate pushdown, so Spark reads only the data it actually needs. CSV/JSON are row-based text formats with no such optimizations, and are much slower and larger at scale.

Q2: What is partition pruning, and how do you enable it? A: Partition pruning is when Spark skips reading entire folders/files that can't match a query's filter, based on the folder structure created by `partitionBy()` (e.g., `year=2024/month=01/`). You enable it by writing partitioned data on columns commonly used in filters, and filtering on those same columns when reading.

Q3: What's the "small file problem" and how does partitioning relate to it? A: Partitioning by a high-cardinality column (like `user_id` or raw timestamp) creates an enormous number of small files. Many tiny files hurt performance due to per-file overhead. Fix: partition by low-cardinality columns (like date at day/month granularity) and periodically compact small files.

Q4: What's the difference between `overwrite` and `append` write modes, and a risk of `overwrite`? A: `overwrite` replaces existing data at the destination; `append` adds new data alongside it. A careless `overwrite` can silently delete valid historical data if the write path

isn't scoped correctly — this is why partition-aware dynamic overwrite is often used in production ETL.

Q5: How would you flatten a JSON array column into one row per array element? A: Use `F.explode("array_column")` in a `select`, turning each array element into its own row while duplicating the other column values.

Module 4 — Spark SQL

4.1 Temp views, `spark.sql()`

Spark lets you register a DataFrame as a **temporary view** and then query it with plain SQL — genuinely useful when a query is easier to express in SQL than chained DataFrame calls:

```
python

df.createOrReplaceTempView("employees")

result = spark.sql("""
    SELECT department, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
""")
result.show()
```

Under the hood, `spark.sql()` and the DataFrame API compile down to the **same** logical plan and go through the **same** Catalyst optimizer — so there's no performance difference. Use whichever is more readable for the task.

4.2 Window functions (`rank`, `row_number`, `lead/lag`, `running totals`)

Window functions compute a value **across a group of rows related to the current row**, without collapsing rows the way `groupBy` does (each input row still gets an output row).

```
python
```

```

from pyspark.sql import Window
from pyspark.sql import functions as F

window_spec = Window.partitionBy("department").orderBy(F.col("salary").desc())

df.withColumn("rank", F.rank().over(window_spec)) \
  .withColumn("row_number", F.row_number().over(window_spec)) \
  .withColumn("prev_salary", F.lag("salary", 1).over(window_spec)) \
  .withColumn("next_salary", F.lead("salary", 1).over(window_spec)) \
  .show()

# Running total (unbounded preceding to current row)
running_window = Window.partitionBy("department").orderBy("date") \
  .rowsBetween(Window.unboundedPreceding, Window.currentRow)
df.withColumn("running_total", F.sum("amount").over(running_window)).show()

```

`rank()` vs `row_number()` vs `dense_rank()`: with a tie (two rows with the same salary), `row_number()` gives them different numbers (arbitrary order), `rank()` gives them the same rank but **skips** the next number (1,1,3), `dense_rank()` gives them the same rank with **no gap** (1,1,2).

4.3 UDFs vs built-in functions (and why to avoid UDFs when possible)

A **UDF (User Defined Function)** lets you write custom Python logic and apply it to a column. But regular Python UDFs are **slow**: for every row, data has to be serialized out of Spark's optimized JVM memory format, sent to a separate Python process, processed, and serialized back. This "row-by-row Python roundtrip" kills performance, and it also makes the function opaque to Catalyst (no optimization possible).

```

python

from pyspark.sql.functions import udf
from pyspark.sql.types import IntegerType

@udf(returnType=IntegerType())
def add_one(x):
    return x + 1

df.withColumn("x_plus_1", add_one(df.x))

```

Golden rule: always check if a built-in function (`pyspark.sql.functions`) already does what you need — built-ins run inside the JVM, fully optimized, no serialization overhead. Only reach for a UDF when there's truly no built-in equivalent.

4.4 Pandas UDFs (vectorized UDFs)

Pandas UDFs (built on Apache Arrow) process data in **batches** as pandas Series, instead of row-by-row — dramatically faster than regular UDFs while still letting you use arbitrary Python/pandas/numpy logic.

```
python

import pandas as pd
from pyspark.sql.functions import pandas_udf

@pandas_udf("double")
def add_one_pandas(x: pd.Series) -> pd.Series:
    return x + 1

df.withColumn("x_plus_1", add_one_pandas(df.x))
```

Rule of thumb: built-in functions > Pandas UDFs > regular (row-at-a-time) UDFs, in order of preference.

Practice Exercises — Module 4

1. Register a DataFrame of orders as a temp view and write a SQL query to find the top 3 customers by total spend.
2. Using window functions, rank employees within each department by salary, and also compute each employee's salary difference from the next-highest-paid person in their department (hint: `lag()`/`lead()`).
3. Compute a running total of daily sales per store using `rowsBetween`.
4. Write a regular Python UDF that converts Fahrenheit to Celsius, then rewrite it as a Pandas UDF. (No need to benchmark unless you want to — just understand the code difference.)

Interview Q&A — Module 4

Q1: Is there a performance difference between `spark.sql("...")` and the equivalent DataFrame API calls? A: No — both compile to the same logical plan and go through the same Catalyst optimizer. The choice is purely about readability/preference, not performance.

Q2: Explain the difference between `rank()`, `dense_rank()`, and `row_number()`. A: All three assign an ordering number within a window. `row_number()` always gives unique, sequential numbers even for ties. `rank()` gives tied rows the same number but leaves a gap afterward (1,1,3). `dense_rank()` gives tied rows the same number with no gap (1,1,2).

Q3: Why are regular Python UDFs slow in Spark, and what's the alternative? A: Regular UDFs process one row at a time and require serializing data out of Spark's JVM-optimized format into Python and back for every row — this overhead dominates runtime, and

Catalyst can't optimize inside a UDF. The alternatives, in order of preference: use a built-in Spark SQL function if one exists; otherwise use a Pandas UDF (Arrow-based, vectorized/batch processing), which is much faster than a row-at-a-time UDF.

Q4: What does `Window.partitionBy("dept").orderBy("salary")` actually define? A: It defines a "window" of rows — grouped by department, ordered by salary within each group — over which a window function (like `rank()`, `sum()`, `lag()`) computes its result for each row, without collapsing the rows the way a `groupBy().agg()` would.

Q5: How would you compute a running total in Spark SQL / DataFrame API? A: Use a window function with a frame from `unboundedPreceding` to `currentRow`:

```
F.sum("amount").over(Window.partitionBy(...).orderBy(...).rowsBetween(Window.unboundedPreceding, Window.currentRow))
```

Module 5 — Performance & Internals

5.1 Catalyst optimizer & physical plans (`explain()`)

Catalyst is Spark SQL's query optimizer. When you build a chain of DataFrame/SQL operations, Spark doesn't just run them in the order you wrote them — it goes through several stages:

1. **Unresolved logical plan** — your code, parsed but not yet validated against actual table/column names.
2. **Resolved (analyzed) logical plan** — column/table names verified against the catalog.
3. **Optimized logical plan** — Catalyst applies rule-based optimizations: predicate pushdown (move filters as close to the data source as possible), column pruning (drop unused columns early), constant folding, etc.
4. **Physical plan(s)** — several possible ways to actually execute the plan (e.g., which join algorithm to use); Spark picks the cheapest one using a cost model.

You can inspect this yourself:

```
python
df.filter(df.age > 18).select("name").explain(True)
```

`explain(True)` (or `explain(mode="extended")`) shows all four stages — reading physical plans is a genuinely valuable skill for debugging slow queries and is a favorite interview topic.

5.2 Partitions, repartition vs coalesce

A Spark **partition** is a chunk of data that gets processed by a single task on a single executor core. More partitions = more parallelism, but too many small partitions adds scheduling overhead; too few large partitions underuses the cluster.

- `repartition(n)` — reshuffles data across the cluster to create exactly `n` partitions (can increase or decrease partition count). This involves a full shuffle (expensive) but gives evenly-sized partitions.
- `coalesce(n)` — reduces the number of partitions **without** a full shuffle, by combining existing partitions. Much cheaper, but can result in uneven partition sizes, and it can only *decrease* partition count (not increase).

python

```
df.repartition(200)          # expensive, even distribution, can go up or down
df.coalesce(10)             # cheap, combines partitions, can only go down
```

Common pattern: use `coalesce()` right before writing output, to avoid creating too many small output files, without paying for a full shuffle.

5.3 Shuffles — what causes them, how to reduce them

A **shuffle** is when data must be physically moved between executors across the network — e.g., all rows with the same key need to end up on the same executor for a `groupBy` or a join. Shuffles are the single biggest performance cost in Spark: they involve disk I/O, network transfer, and serialization.

Operations that typically cause shuffles: `groupBy`, `join` (non-broadcast), `distinct`, `repartition`, `orderBy`.

Ways to reduce shuffle cost:

- Filter and select only needed columns *before* a shuffle-heavy operation (less data to move).
- Use broadcast joins for small tables (see below — avoids shuffling the large table entirely).
- Avoid unnecessary `repartition()` calls.
- Pre-partition data on disk (e.g., by join key) if the same join happens repeatedly.

5.4 Broadcast joins vs shuffle joins

- **Shuffle join (sort-merge join)** — default for joining two large tables. Both tables get shuffled so matching keys land on the same executor. Expensive but necessary when both sides are large.
- **Broadcast join** — if one table is small enough to fit in memory, Spark sends (broadcasts) a full copy of it to *every* executor, so the large table never needs to be shuffled at all — only the large table is scanned once, locally, on each executor.

python

```
from pyspark.sql.functions import broadcast

df_large.join(broadcast(df_small), on="id")
```

Spark actually does this **automatically** for small tables under

`spark.sql.autoBroadcastJoinThreshold` (default 10 MB) — the explicit `broadcast()` hint is for when you know a table is small but Spark's size estimate is wrong or unavailable.

5.5 Caching/persistence strategies

If you reuse the same DataFrame multiple times (e.g., in a loop, or in several downstream branches), Spark will **recompute it from scratch each time** unless you cache it.

```
python

df.cache()           # shorthand for persist(MEMORY_AND_DISK)
df.persist(StorageLevel.MEMORY_ONLY)
df.count()           # trigger the actual caching (cache is also lazy!)
...
df.unpersist()       # free the memory when done
```

Storage levels trade off memory usage vs. recomputation cost: `MEMORY_ONLY` (fastest, but data lost if it doesn't fit and gets evicted), `MEMORY_AND_DISK` (spills to disk if it doesn't fit, safer default), `DISK_ONLY`, and `*_SER` variants (serialized — more compact but slightly slower to access).

Rule of thumb: cache only when a DataFrame is reused multiple times — caching something used only once just adds overhead for no benefit.

5.6 Skew handling (salting techniques)

Data skew happens when one or a few keys have vastly more rows than others (e.g., one `customer_id` has 40% of all transactions). During a shuffle-based `groupBy`/join, all rows for that one key go to a single task/executor — that one task becomes a massive bottleneck while other executors sit idle.

Salting is a technique to fix skew in joins/aggregations: artificially split a skewed key into several "sub-keys" by appending a random number, spreading its rows across multiple tasks, then combining results afterward.

```
python
```

```
import random
from pyspark.sql import functions as F

num_salts = 10
df_salted = df.withColumn("salt", (F.rand() * num_salts).cast("int"))
df_salted = df_salted.withColumn("salted_key", F.concat_ws("_", "key", "salt"))
# join/group on salted_key, then aggregate again to combine salt groups back to
```

In modern Spark (3.x+), **Adaptive Query Execution (AQE)** (Module 9) can automatically detect and handle skewed joins for you, reducing (but not eliminating) the need for manual salting.

Practice Exercises — Module 5

1. Take a multi-step DataFrame pipeline (filter + groupBy + join) and call `.explain(True)` on it. Try to identify the predicate pushdown in the optimized logical plan.
2. Create two DataFrames of very different sizes, join them, and use `.explain()` to check whether Spark chose a broadcast join automatically. Force a broadcast join explicitly with `broadcast()`.
3. Experiment with `repartition(n)` vs `coalesce(n)` on a DataFrame and inspect `df.rdd.getNumPartitions()` before/after each.
4. Cache a DataFrame that's reused for two different aggregations, and compare timing with and without `.cache()`.

Interview Q&A — Module 5

Q1: What is a shuffle, and why is it expensive? A: A shuffle is when Spark redistributes data across the network so that rows with matching keys land on the same executor (needed for operations like `groupBy`, non-broadcast `join`, `distinct`). It's expensive because it involves disk writes, network transfer, and serialization/deserialization across potentially hundreds of tasks — it's usually the dominant cost in a Spark job.

Q2: What's the difference between `repartition()` and `coalesce()`? A: `repartition(n)` performs a full shuffle to redistribute data into exactly `n` evenly-sized partitions, and can increase or decrease the partition count. `coalesce(n)` merges existing partitions without a full shuffle (cheaper), but can only decrease partition count and may produce unevenly sized partitions.

Q3: When does Spark use a broadcast join, and why is it faster than a shuffle join? A: Spark broadcasts a table automatically when its estimated size is below `spark.sql.autoBroadcastJoinThreshold` (default 10MB), or when explicitly hinted with `broadcast()`. It's faster because the small table is sent once to every executor, letting the large table be joined locally without ever being shuffled — avoiding the network/disk cost of shuffling the large side.

Q4: What is data skew, and how does salting fix it? A: Data skew is when a small number of keys have disproportionately many rows, causing the tasks handling those keys to become bottlenecks while others sit idle. Salting appends a random suffix to skewed keys to artificially split them into multiple sub-keys, spreading the load across more tasks; results are then re-aggregated across the salt values.

Q5: You call `.cache()` on a DataFrame but don't see any performance benefit. What might be wrong? A: A few common causes: (1) caching is lazy — nothing is actually cached until an action runs on the DataFrame after `.cache()` is called; (2) the DataFrame is only used once, so there's nothing to "reuse" — caching adds overhead with no payoff; (3) the DataFrame is too large to fit in memory and is being evicted/recomputed anyway — check the Spark UI's Storage tab.

Module 6 — Data Engineering Patterns

6.1 Building idempotent, incremental ETL pipelines

Idempotent means: running the same job twice with the same input produces the same result — no duplicated or corrupted data. This matters enormously because real pipelines fail and get **re-run** (retries, backfills, manual re-runs after a bug fix).

Common techniques:

- Write with `overwrite` on a specific partition, rather than `append`, when reprocessing a day's data (re-running the job for "2024-01-15" should replace, not duplicate, that day's output).
- Use a **MERGE** (upsert) operation (Delta Lake, below) instead of blind `append` for incremental loads — match on a business key so re-running doesn't create duplicates.
- Track "high-water marks" (e.g., last processed timestamp/ID) so **incremental** loads only pull new/changed data since the last successful run — instead of reprocessing the entire source every time.

6.2 Slowly Changing Dimensions (SCD Type 1 & 2) in Spark

SCDs handle how dimension tables (e.g., "customers", "products") deal with attribute changes over time.

- **SCD Type 1** — overwrite the old value; no history kept. E.g., customer changes their address → just update the row.

```
python
```

```
# Simplest form: just overwrite the dimension table with latest values
new_dim.write.mode("overwrite").saveAsTable("dim_customer")
```

- **SCD Type 2** — keep full history: insert a new row for the change, and mark old rows as expired using `effective_date`/`end_date`/`is_current` columns.

python

```
# Conceptually:  
# 1. Find changed records (compare source to current "is_current=True" rows)  
# 2. Expire old rows: set end_date = today, is_current = False  
# 3. Insert new rows with start_date = today, end_date = null, is_current = True
```

In practice, SCD Type 2 is almost always implemented via Delta Lake's `MERGE INTO` (below) rather than hand-written logic, because MERGE handles the match/update/insert logic atomically.

6.3 Data quality checks & validation

Before trusting data downstream, validate it. Common checks:

- **Null checks** — are required columns actually non-null?
`((df.filter(F.col("id").isNull()).count()))`
- **Uniqueness checks** — are primary keys actually unique? `((df.count() == df.select("id").distinct().count()))`
- **Range/type checks** — is `age` between 0 and 120? Is `amount` non-negative?
- **Row count sanity checks** — did today's row count fall suspiciously compared to the historical average? (catches upstream pipeline failures)
- **Referential integrity** — do foreign keys actually exist in the referenced table? (an anti-join finds violations)

In production, tools like **Great Expectations** or **Delta Live Tables expectations** formalize these checks; conceptually they're all just DataFrame filters/counts.

6.4 Delta Lake: ACID transactions, time travel, MERGE INTO, schema evolution

Delta Lake is a storage layer on top of Parquet that adds database-like reliability features to your data lake:

- **ACID transactions** — writes are all-or-nothing; concurrent readers never see a half-written table. This solves a real Spark/Parquet pain point: a job crashing mid-write used to leave the table in a corrupted, partially-written state.
- **Time travel** — query previous versions of a table:

python

```
spark.read.format("delta").option("versionAsOf", 5).load("path")  
spark.read.format("delta").option("timestampAsOf", "2024-01-01").load("path")
```

- **MERGE INTO** — the core tool for upserts (insert-or-update) and SCD Type 2:

python


```

from delta.tables import DeltaTable

target = DeltaTable.forPath(spark, "path/to/delta_table")
target.alias("t").merge(
    source_df.alias("s"),
    "t.id = s.id"
).whenMatchedUpdateAll() \
  .whenNotMatchedInsertAll() \
  .execute()

```

- **Schema evolution** — allow new columns to be added automatically on write, instead of failing:

python

```
df.write.format("delta").mode("append").option("mergeSchema", "true").save("pat
```

Delta Lake has become the de facto standard for reliable data lakes/lakehouses (used heavily in Databricks) — expect interview questions here.

6.5 Medallion architecture (Bronze/Silver/Gold)

A widely used pattern for organizing a lakehouse into progressive layers of data quality/refinement:

- **Bronze** — raw data, ingested as-is from source systems, minimal/no transformation (preserves the original for reprocessing/auditing).
- **Silver** — cleaned, validated, conformed data — deduplicated, typed correctly, joined to reference data, ready for general analysis.
- **Gold** — business-level aggregates and curated tables, optimized for specific reporting/ML use cases (e.g., "daily_revenue_by_region").

Data generally flows Bronze → Silver → Gold, with data quality and business logic increasing at each stage.

Practice Exercises — Module 6

1. Simulate re-running an ETL job twice on the same day's data using `overwrite` mode on a partition — confirm no duplicates result. Then try it with `append` mode and see the duplication problem firsthand.
2. Implement a simple SCD Type 2 table by hand for a small "customers" dataset with an address change, using plain DataFrame logic (expire old row, insert new row).
3. Install `delta-spark` locally, create a small Delta table, and practice a `MERGE INTO` upsert.

4. Write a small data-quality check function that reports null counts, duplicate primary keys, and out-of-range values for a given DataFrame and column list.

Interview Q&A — Module 6

Q1: What does "idempotent" mean in the context of an ETL pipeline, and why does it matter? A: An idempotent pipeline produces the same result no matter how many times it's re-run with the same input — no duplicated or corrupted data. It matters because production jobs get retried and backfilled; a non-idempotent pipeline silently creates duplicate or inconsistent data every time it's re-run.

Q2: Explain the difference between SCD Type 1 and Type 2, with an example. A: SCD Type 1 overwrites the old value with no history (e.g., a customer's address is simply updated in place). SCD Type 2 preserves history by inserting a new row for the change and marking the old row as expired (using `effective_date`/`end_date`/`is_current` flags), so you can answer "what was this customer's address on March 1st?"

Q3: What problem does Delta Lake solve that plain Parquet doesn't? A: Plain Parquet has no transactional guarantees — a failed write mid-job can leave a table partially written/corrupted, and there's no built-in way to do atomic upserts, see historical versions, or safely evolve schema. Delta Lake adds ACID transactions, `MERGE INTO` for upserts, time travel to old versions, and safe schema evolution on top of Parquet files.

Q4: Describe the Bronze/Silver/Gold (Medallion) architecture. A: Bronze holds raw, as-ingested data for auditability. Silver holds cleaned, validated, deduplicated, conformed data. Gold holds curated, business-level aggregates optimized for specific reporting or ML consumption. Data quality and business logic increase progressively through the layers.

Q5: How would you use MERGE INTO to implement an upsert (insert new records, update existing ones) in Delta Lake? A: Match source and target on a business key (e.g., `t.id = s.id`), then chain `.whenMatchedUpdateAll()` to update existing matched rows and `.whenNotMatchedInsertAll()` to insert new unmatched rows, executed atomically in a single `MERGE` statement.

Module 7 — Structured Streaming

7.1 Micro-batch vs continuous processing

Spark Structured Streaming primarily works in **micro-batch** mode: instead of processing each event the instant it arrives (true continuous/row-at-a-time processing), Spark collects incoming data into small batches (e.g., every 1 second, or as fast as possible) and processes each batch using the same DataFrame engine as batch jobs. This gives you the exact same API for batch and streaming, at the cost of small (sub-second to a few seconds) latency vs. true continuous processing.

Spark does offer an **experimental continuous processing mode** for ultra-low latency, but it supports a much smaller set of operations and is rarely used in typical data engineering — micro-batch is the standard.

7.2 Reading streams (files, Kafka)

python

```
# Reading a stream of new files landing in a directory
stream_df = (
    spark.readStream
        .schema(my_schema)    # streaming sources require an explicit schema
        .format("json")
        .load("path/to/incoming/")
)

# Reading from Kafka
kafka_df = (
    spark.readStream
        .format("kafka")
        .option("kafka.bootstrap.servers", "host:9092")
        .option("subscribe", "my_topic")
        .load()
)
# Kafka messages arrive as binary key/value – you typically cast and parse them
from pyspark.sql.functions import col
parsed = kafka_df.selectExpr("CAST(value AS STRING)")
```

Writing a stream out looks like this:

python

```
query = (
    stream_df.writeStream
        .format("parquet")
        .option("checkpointLocation", "path/to/checkpoint")
        .outputMode("append")
        .start("output_path")
)
query.awaitTermination()
```

7.3 Watermarking, windowed aggregations

Windowed aggregation — group streaming events into time buckets (e.g., "count events per 5-minute window") instead of a single running total:

python

```
from pyspark.sql.functions import window

agg = (
    stream_df
    .groupBy(window("event_time", "5 minutes"))
    .count()
)
```

Watermarking — streaming data can arrive **late** (network delays, retries). Spark needs to know how long to wait for late data before finalizing a window's result and discarding related state (otherwise it would have to keep state forever, which isn't feasible). A watermark declares: "I'll tolerate data up to N minutes late; after that, I'll finalize the window and drop state for it."

python

```
agg = (
    stream_df
    .withWatermark("event_time", "10 minutes") # tolerate up to 10 min late d
    .groupBy(window("event_time", "5 minutes"))
    .count()
)
```

7.4 Streaming joins, checkpointing, fault tolerance

- **Streaming joins** — you can join a stream to a static (batch) DataFrame, or join two streams together (stream-stream joins require watermarks on both sides, since Spark needs to know when it's safe to stop waiting for a match).
- **Checkpointing** — Structured Streaming periodically saves its progress (which offsets/data it has processed, plus intermediate aggregation state) to a **checkpoint location** (durable storage like cloud storage/HDFS). If the job crashes and restarts, it resumes exactly where it left off, using the checkpoint — this is the mechanism behind Spark's **exactly-once** processing guarantees for supported sinks.
- **Fault tolerance** — combined with checkpointing, Spark tracks source offsets (e.g., Kafka offsets) so a restarted query re-processes only what's needed and doesn't duplicate or lose data, as long as the source supports replay (Kafka does; a plain socket source does not).

Practice Exercises — Module 7

1. Set up a local "file stream": write small JSON files one at a time into a folder, and run a `readStream` job that counts rows and prints results with `outputMode("complete")`.
2. Add a `window("event_time", "1 minute")` aggregation to the above and simulate a few events with different timestamps.

3. Add `withWatermark` and simulate a "late" event arriving after the watermark threshold — observe that it gets dropped.
4. Kill and restart your streaming query mid-run (with a checkpoint location configured) and confirm it resumes without reprocessing old data.

🌟 Interview Q&A — Module 7

Q1: What is micro-batch processing, and how does it differ from true continuous processing? A: Micro-batch processing collects incoming streaming data into small batches processed at short intervals, reusing the same batch DataFrame engine — giving unified batch/streaming APIs at the cost of small latency. True continuous (row-at-a-time) processing has lower latency but is a separate, more limited experimental mode in Spark, rarely used in practice.

Q2: What is a watermark, and why is it necessary in streaming aggregations? A: A watermark tells Spark how long to tolerate late-arriving data before finalizing a windowed aggregation and discarding its state. It's necessary because Spark can't keep state for every window forever (unbounded memory growth) — the watermark defines a cutoff for "how late is too late."

Q3: What is checkpointing used for in Structured Streaming? A: Checkpointing durably saves a streaming query's progress — which data/offsets have been processed and intermediate aggregation state — so that if the job fails and restarts, it resumes exactly where it left off, without reprocessing or losing data (enabling exactly-once semantics for supported sinks).

Q4: Why do stream-stream joins require watermarks on both sides? A: Spark must know how long to wait for a matching event from the other stream before giving up and considering a row "unmatched." Without a watermark, Spark would need to buffer all historical data from both streams indefinitely, which isn't feasible.

Q5: What's a real difference between reading from a file source vs. a Kafka source for streaming, in terms of replay/fault tolerance? A: Kafka retains messages and exposes offsets, so a restarted Spark job can precisely resume from the last processed offset — a core requirement for exactly-once guarantees. A source that doesn't support replay (like a raw TCP socket) can't guarantee this, since once data is read it's gone.

Module 8 — Production & Orchestration

8.1 Cluster sizing & configuration tuning

Key configs to understand (not memorize exact numbers — understand the trade-offs):

- `spark.executor.memory` — memory per executor. Too low → out-of-memory errors/spilling to disk; too high → fewer executors fit per machine, wasting parallelism.
- `spark.executor.cores` — CPU cores per executor (controls how many tasks run in parallel within one executor). A common guideline is 4-5 cores per executor (too many cores per executor can cause HDFS I/O contention and reduce throughput).

- `spark.sql.shuffle.partitions` — number of partitions used after a shuffle (default 200). For small data, 200 is often too many (lots of tiny tasks); for huge data, it can be too few. This is one of the most commonly tuned settings.
- `spark.driver.memory` — memory for the driver; needs to be large if you `.collect()` big result sets to the driver (generally avoid doing that at all).

8.2 Job orchestration (Airflow / Databricks Workflows)

Real pipelines are rarely "run one script." They're **DAGs of dependent steps** — e.g., "ingest raw data" → "run data quality checks" → "build silver table" → "build gold aggregates" → "trigger downstream BI refresh," with retries, alerting, and scheduling.

- **Apache Airflow** — a general-purpose Python-based orchestrator. You define a DAG of tasks (often using a `SparkSubmitOperator` or a Databricks operator) with dependencies, schedules, and retry policies.
- **Databricks Workflows** — a built-in orchestrator inside Databricks specifically for scheduling and chaining notebooks/Spark jobs, with less setup overhead if you're already on Databricks.

Either way, the underlying idea is the same: express your pipeline as a dependency graph, not a single monolithic script, so pieces can be retried, monitored, and reasoned about independently.

8.3 Monitoring via Spark UI

The **Spark UI** (available while a job runs, and often archived afterward as the "Spark History Server") is essential for debugging performance issues:

- **Jobs tab** — see all Spark jobs (triggered by each action), their stages, and duration.
- **Stages tab** — see tasks within a stage; look for **skew** (a few tasks taking far longer than others — a red flag for data skew) and **spill** (data too big for memory, spilled to disk — a sign you need more memory or fewer/larger partitions).
- **SQL tab** — visual representation of the physical query plan, with metrics per operator (rows processed, time spent) — extremely useful for finding the expensive step in a query.
- **Storage tab** — see what's cached and how much memory it's using.
- **Executors tab** — per-executor resource usage (memory, task time, shuffle read/write) — useful for spotting an unevenly loaded cluster.

8.4 CI/CD for Spark jobs, testing with pytest + chispa/pyspark-test

Spark code should be unit tested like any other code:

```
python
```

```
# test_transformations.py
import pytest
from pyspark.sql import SparkSession
from chispa import assert_df_equality
from my_pipeline import add_full_name # your transformation function

@pytest.fixture(scope="session")
def spark():
    return SparkSession.builder.master("local[2]").appName("tests").getOrCreate()

def test_add_full_name(spark):
    input_df = spark.createDataFrame([("John", "Doe")], ["first", "last"])
    result_df = add_full_name(input_df)
    expected_df = spark.createDataFrame([("John", "Doe", "John Doe")], ["first", "last", "full_name"])
    assert_df_equality(result_df, expected_df, ignore_row_order=True)
```

Key ideas:

- Write your pipeline logic as small, pure **functions that take and return DataFrames**, so they're independently testable (rather than one giant script).
- `chispa` (or `pyspark-test`) provides DataFrame-equality assertions with helpful diffs, since comparing DataFrames isn't as simple as `==`.
- CI (e.g., GitHub Actions) runs these tests automatically on every pull request, using a small local Spark session — no real cluster needed for unit tests.

8.5 Cost optimization on cloud platforms

- **Right-size clusters** — don't over-provision; use autoscaling clusters that grow/shrink with workload instead of fixed large clusters running 24/7.
- **Spot/preemptible instances** — for fault-tolerant batch jobs, use much cheaper spot instances for executors (with the driver on stable on-demand infrastructure).
- **Use columnar formats + partitioning** (Parquet/Delta with sensible partitioning) — directly reduces the amount of data scanned, which reduces both time and (in cloud data-scanning pricing models) cost.
- **Cluster auto-termination** — shut down idle interactive clusters automatically (a very common Databricks cost leak is a cluster left running overnight).
- **Job clusters vs. all-purpose clusters** — use ephemeral job-specific clusters for scheduled production jobs (spun up, run, torn down) rather than sharing a persistent interactive cluster, which is usually more expensive per unit of actual work done.

Practice Exercises — Module 8

1. Run a Spark job locally with a deliberately small `spark.executor.memory`/small shuffle partitions setting, and observe (via Spark UI or logs) disk spill happening. Then re-run with better settings and compare.

2. Open the Spark UI (`localhost:4040`) while a local job runs) for a job with a join and a `groupBy`, and identify the shuffle stages in the "Stages" tab.
3. Write a small pipeline as a pure function (`def clean_orders(df): ...`) and write a `chispa`-based unit test for it.
4. Sketch (on paper or in a README) what an Airflow DAG for a Bronze → Silver → Gold pipeline would look like: task names and dependencies.

Interview Q&A — Module 8

Q1: What does `spark.sql.shuffle.partitions` control, and why is the default of 200 often wrong? A: It controls how many partitions are created after a shuffle (e.g., after a `groupBy` or join). The default of 200 is a fixed number regardless of data size — for small datasets it creates too many tiny, overhead-heavy tasks; for very large datasets it can create too few, overly large partitions that don't parallelize well. It's commonly tuned based on data size (or handled automatically by AQE in modern Spark).

Q2: Why should Spark transformation logic be written as small, pure functions rather than one long script? A: Pure functions that take a `DataFrame` and return a `DataFrame` can be unit tested in isolation (with a local `SparkSession` and small sample data), reused across pipelines, and reasoned about independently — a monolithic script is hard to test and debug piece by piece.

Q3: In the Spark UI, what would you look for to diagnose a data skew problem? A: In the Stages tab, look at the task duration distribution for a stage — skew shows up as most tasks finishing quickly while one or a few tasks (handling a disproportionately large key) take far longer than the rest, dragging out the whole stage.

Q4: Why use ephemeral "job clusters" instead of a shared "all-purpose cluster" for scheduled production Spark jobs? A: Job clusters spin up only for the duration of the job and terminate afterward, so you pay only for the compute actually used. Shared all-purpose/interactive clusters tend to run continuously (or be left running idle) and are typically priced higher per unit of compute, making them a common source of wasted cloud cost for scheduled workloads.

Q5: What's the difference between using spot/preemptible instances vs. on-demand instances for a Spark cluster, and what's the trade-off? A: Spot/preemptible instances are much cheaper but can be reclaimed by the cloud provider at any time, causing executor loss. This is fine for fault-tolerant batch jobs (Spark can recompute lost work, especially with checkpointing), so executors are often run on spot while the driver (a single point of failure for the whole job) runs on stable on-demand infrastructure.

Module 9 — Advanced / Interview-Ready Topics

9.1 Custom partitioners, custom data sources

Custom partitioners (mostly an RDD-level concept) let you control exactly how data is distributed across partitions based on your own logic, instead of Spark's default hash

partitioning — useful when you know something about your data's key distribution that Spark doesn't (e.g., co-locating related keys to avoid a shuffle in a later join).

```
python
```

```
rdd.partitionBy(10, partitionFunc=my_custom_partition_function)
```

Custom data sources — Spark's Data Source API (V2) lets you build connectors for systems Spark doesn't natively support (a proprietary database, an internal API, etc.), implementing how Spark should read/write batches (and optionally streams) from that system. Most data engineers *use* existing connectors far more often than they *build* custom ones, but understanding that this extensibility exists — and roughly how catalog/read/write interfaces plug in — is a good advanced-interview signal.

9.2 Adaptive Query Execution (AQE)

AQE (default-on since Spark 3.2) lets Spark **re-optimize a query plan mid-execution**, using real runtime statistics instead of only pre-execution estimates. Three big things AQE does automatically:

1. **Dynamically coalescing shuffle partitions** — merges many small post-shuffle partitions into fewer, right-sized ones, instead of you having to hand-tune `spark.sql.shuffle.partitions`.
2. **Dynamically switching join strategies** — if a table turns out to be smaller than expected at runtime (e.g., after a filter reduced it a lot), Spark can switch a planned shuffle join into a broadcast join on the fly.
3. **Dynamically optimizing skewed joins** — detects a skewed partition at runtime and automatically splits it into smaller sub-partitions (similar in spirit to manual salting, but automatic).

```
python
```

```
spark.conf.set("spark.sql.adaptive.enabled", "true") # default True in modern
```

AQE doesn't eliminate the need to understand shuffles/skew/joins — it just means Spark can self-correct in many common cases where its initial (pre-execution) estimates were wrong.

9.3 Memory management deep dive (execution vs. storage memory)

Within each executor's JVM heap, Spark's **Unified Memory Manager** splits usable memory into two dynamically-shared regions:

- **Execution memory** — used for computation: shuffles, joins, sorts, aggregations. Temporary — released once the task finishes.
- **Storage memory** — used for caching DataFrames/RDDs (`.cache()`/`.persist()`) and broadcast variables.

These two regions can **borrow from each other** dynamically (unified memory management, since Spark 1.6+): if storage isn't using its share, execution can borrow it (and vice versa), rather than having two hard, wasted-if-unused partitions like in older Spark versions. However, cached blocks can be evicted under memory pressure from execution tasks (execution memory generally takes priority, since a task failing due to lack of execution memory is worse than losing a cached block that can be recomputed).

Beyond the heap, Spark also uses **off-heap memory** (optional, `(spark.memory.offHeap.enabled)`) for some operations, and there's separate **overhead memory** reserved for JVM internals, network buffers, etc.

`(spark.executor.memoryOverhead)` — a very common cause of executor OOM/container-killed errors is under-provisioning this overhead, not the main executor memory.

9.4 Common data engineering interview problems solved in PySpark

Below are classic problems interviewers use to test practical PySpark fluency, with the core approach for each.

a) Find the second-highest salary per department.

```
python

from pyspark.sql import Window
from pyspark.sql import functions as F

w = Window.partitionBy("department").orderBy(F.col("salary").desc())
result = (
    df.withColumn("rnk", F.dense_rank().over(w))
      .filter(F.col("rnk") == 2)
)
```

b) Find duplicate rows based on a subset of columns.

```
python

from pyspark.sql import Window
w = Window.partitionBy("email").orderBy("id")
df.withColumn("rn", F.row_number().over(w)).filter(F.col("rn") > 1) # the actu
```

c) Word count (the "hello world" of distributed data processing).

```
python

words = df.select(F.explode(F.split(F.col("line"), " ")).alias("word"))
word_counts = words.groupBy("word").count().orderBy(F.desc("count"))
```

d) Find sessions from raw event logs (gap-based sessionization).

python

```
w = Window.partitionBy("user_id").orderBy("event_time")
df = df.withColumn("prev_time", F.lag("event_time").over(w))
df = df.withColumn(
    "new_session",
    F.when(
        (F.col("prev_time").isNull()) |
        (F.col("event_time").cast("long") - F.col("prev_time").cast("long") > 1
         1
        ).otherwise(0)
    )
df = df.withColumn("session_id", F.sum("new_session").over(w.rowsBetween(Window
```

e) Pivot data (rows to columns).

python

```
df.groupBy("department").pivot("year").agg(F.sum("revenue"))
```

f) Find the top N records per group.

python

```
w = Window.partitionBy("category").orderBy(F.col("sales").desc())
df.withColumn("rn", F.row_number().over(w)).filter(F.col("rn") <= 3) # top 3 p
```

Practice Exercises — Module 9

1. Solve the "second-highest salary per department" problem yourself (without looking at the solution above first), using a sample dataset you create.
2. Given a raw event log with `user_id` and `event_time`, implement gap-based sessionization (define a session as ending after 30 minutes of inactivity).
3. Enable/disable `spark.sql.adaptive.enabled` and compare `.explain()` output / execution behavior on a join between a very large and a moderately sized table where the moderate table is filtered down significantly before the join.
4. Write a query that finds the top 2 highest-selling products per category from a sales DataFrame.

Interview Q&A — Module 9

Q1: What three main optimizations does Adaptive Query Execution (AQE) perform at runtime? A: (1) Dynamically coalescing shuffle partitions into right-sized ones instead of relying on a fixed `spark.sql.shuffle.partitions`; (2) dynamically switching a shuffle join

to a broadcast join if a table turns out smaller than the initial estimate at runtime; (3) dynamically detecting and splitting skewed partitions during a join.

Q2: What's the difference between execution memory and storage memory in Spark, and how are they related?

A: Execution memory is used for active computation (shuffles, joins, sorts, aggregations) and is released when a task finishes. Storage memory is used for cached DataFrames and broadcast variables. Since Spark 1.6, they share a "unified" memory pool and can dynamically borrow from each other, though execution generally takes priority — cached data can be evicted under execution memory pressure.

Q3: How would you find the second-highest salary in each department using PySpark?

A: Use a window function partitioned by department, ordered by salary descending, apply `dense_rank()` (not `rank()`, to avoid skipping if there are ties for #1), and filter for `rnk == 2`.

Q4: How would you implement sessionization (grouping user events into sessions based on a time gap) in PySpark?

A: Use a window partitioned by user and ordered by event time; use `lag()` to get each row's previous event time; flag a "new session" whenever the gap since the previous event exceeds your session timeout (or the previous event is null); then take a running sum of that flag over the same window to produce a `session_id` that increments at each new session boundary.

Q5: A Spark executor is being killed with an "out of memory" / container-killed error, but `spark.executor.memory` looks generous. What's a likely overlooked cause?

`spark.executor.memoryOverhead` — the memory reserved outside the JVM heap for things like off-heap buffers, network I/O, and JVM internals — being set too low. Under-provisioned overhead is a very common cause of container kills even when the main heap memory setting looks sufficient.

How to keep going from here

1. **Go in order.** Each module leans on the previous one's vocabulary (e.g., Module 5's shuffle explanation assumes you understand partitions from Module 2/3).
2. **Actually run the code.** Reading `df.withColumn(...)` isn't the same as watching it execute and seeing `.explain()` output — install PySpark locally (`pip install pyspark`) and type every example yourself.
3. **Re-attempt the interview questions without looking, then check your answer** — this is far more effective for retention than just reading the answers.
4. **After Module 6**, install `delta-spark` (`pip install delta-spark`) to actually run the Delta Lake examples.
5. **After Module 8**, move to a free cloud sandbox (Databricks Community Edition) for Module 7 (streaming with Kafka) and realistic cluster-based practice — local mode can't fully demonstrate cluster behavior (multiple executors, real shuffles across machines, cloud storage).
6. When you finish Module 9, you'll have covered everything on the roadmap — at that point, mock interviews (explaining these concepts out loud, or on a whiteboard) are the best next step before real interviews.

